

Refined Genetic Algorithm - Economic Dispatch Example

Gerald B. Sheble Kristin Brittig
Senior Member Student Member
Iowa State University
Ames, Iowa 50010

KEYWORDS: power systems, optimization, genetic algorithms, economic dispatch, mutation, crossover, elitism, mutation prediction, penalty factor, stopping criteria, and interval approximation.

ABSTRACT: A genetic-based algorithm is used to solve an economic dispatch (ED) problem. The algorithm utilizes payoff information of perspective solutions to evaluate optimality. Thus, the constraints of classical LaGrangian techniques on unit curves are eliminated. Using an economic dispatch problem as a basis for comparison, several different techniques which enhance program efficiency and accuracy, such as mutation prediction, elitism, interval approximation and penalty factors, are explored. Two unique genetic algorithms are also compared. The results are verified for a sample problem using a classical technique.

I. INTRODUCTION

A need for optimality exists in the highly nonlinear and computationally difficult Power Systems Analysis environment. Artificial intelligence, unlike strict mathematical methods, has the apparent ability to adapt to nonlinearities and discontinuities found commonly in physical systems [1].

The linearizations and assumptions made in the ED problems present a classical example. The classical approach to solving this problem uses approximations to limit complexity. The loss of accuracy induced by these approximations is not desirable. Genetic algorithms (GA) can alleviate this undesirable trait by reproducing the optimization techniques found in nature. It does not require the strict continuity of classical search techniques, instead it allows non-linearities and discontinuities to appear in the solution space. The application of this algorithm to the ED problem uses the payoff information of an objective function to determine optimality. As a result, any type of unit characteristic cost curve may be used to test for optimality [1].

David Goldberg's Simple Genetic Algorithm (SGA) is a suitable method for solving most optimization problems. The results produced are acceptable, yet still imperfect.

94 WM 199-0 PWRs A paper recommended and approved by the IEEE Power System Engineering Committee of the IEEE Power Engineering Society for presentation at the IEEE/PES 1994 Winter Meeting, New York, New York, January 30 - February 3, 1994. Manuscript submitted December 30, 1992; made available for printing November 29, 1993.

Since loss of accuracy increases cost, especially with respect to the ED problem, it is beneficial to find new techniques which will boost the performance capability of SGA.

Several such techniques are discussed in this paper. After verifying that the prototype SGA program worked, a comparison was made with the results of the Refined Genetic Algorithm (RGA). Elitism was then added to SGA and those results were included in the comparison. After determining that the chances of the aforementioned GA programs converging on the optimal solution was problematic, several modification techniques were applied to both the RGA and the Simple Genetic Algorithm with elitism (SGAE) to try to improve their efficiency and accuracy. The results of how they effected the programs performance are also discussed.

II. GENETIC ALGORITHMS MODELING

Genetic algorithms are essentially search algorithms based on the mechanics of nature and natural genetics. They combine solution evaluation with randomized, structured exchanges of information between solutions to obtain optimality. Genetic algorithms are considered to be robust methods because restrictions on the solution space are not made during the process. The power of this algorithm stems from its ability to exploit historical information structures from previous solution guesses in an attempt to increase performance of future solutions [2].

It is able to exploit this information because GAs maintain a population of parameter set solutions and iterate on the complete population. Each iteration is called a generation. The problem parameter set is represented by a fixed length chromosome string of binary bits (0,1). This string represents a single solution point in the problem space and it consists of genetic material.

Evaluation of the chromosome string is accomplished by decoding the encoded symbols and calculating the objective function for the problem using the decoded parameter set. For this analysis, two types of encoding schemes were used. Optimization of each unit's output is desired, therefore each chromosome string contains an encoding of the solution for the unit generation. The first encoding scheme simply stacks each unit's output value (in MW) structure in series with every other string. The second encoding method uses the same system for representation and decoding as the first, except that the assigned gene structures are embedded within each other throughout the chromosome string. Figure 1 illustrates these encodings [3].

Implementation of an objective function and constraints in a genetic algorithm are realized within the fitness function. It acts as a pseudoobjective function, since it is a raw measure of the solution value. Inclusion of the

constraints in the fitness function requires the introduction of tolerances to satisfy the constraint equations. If the fitness function and encoding are well designed, accuracy will not suffer. [1]

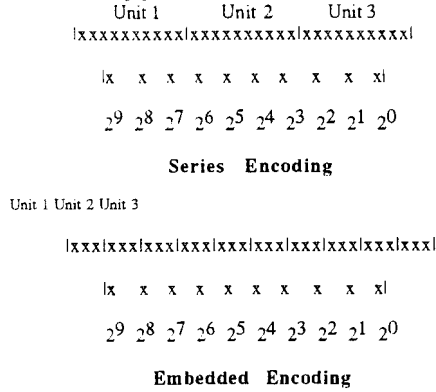


Figure 1: Encoding Methods

Movement in SGA is accomplished using three primary operators: reproduction, mutation, and crossover. The reproduction operator selects a chromosome string from the previous generation based on the string's fitness and its probability of propagation to the next generation. The crossover operator selects a position within two parent strings and swaps the gene information from that position to the end of the string. The mutation operator randomly changes '0's to '1's or vice versa within the bit string.

III. Economic Dispatch

The standard economic dispatch problem can be described mathematically as a single constraint and additive objective function. The single constraint is conservation of energy:

$$\sum P_i + P_L = P_{TL}$$

where

Summation includes all units to be dispatched ($i=1$ to n).

P_i = output generation for unit i

n = number of generators in the system

P_L = total current system load

P_{TL} = total system transmission losses

The objective function is simply the sum of unit costs:

$$OBJ = \sum F_i(P_i)$$

where:

OBJ = objective cost function

F_i = cost function for unit i

The economic dispatch problem is further constrained by each unit's capability:

$$P_i^{\min} \leq P_i \leq P_i^{\max}$$

where

$P_i^{\min}$ = minimum operating output of unit i

$P_i^{\max}$ = maximum operation output of unit i

The solution is restricted to the current on-line generating units only [4].

Input-Output Curves

The generator cost function is obtained from data points taken during "heat run" tests, when input and output data is measured as the unit is slowly varied through its operating region [5]. Smooth quadratic function approximations of the unit input-output characteristics provide the basis for most classical economic dispatch techniques. This approach is used in this analysis.

IV. EXAMPLE APPLICATION

A genetic-based program has been designed to solve economic dispatch problems. A three unit test system was obtained and used in the development of the program.

Unit Data

The example system contains a set of three generators in a simple six bus power system [6]. For values of unit generation P_i , quadratic input-output curve data was obtained using the following equation for each generator:

$$F_i(P_i) = a_i P_i^2 + b_i P_i + c_i$$

Incremental cost curve data was obtained by taking the derivative of the unit input-output equation (above) resulting in the following equation for each generator:

$$\frac{dF_i(P_i)}{dP_i} = 2 a_i P_i + b_i$$

The constraints for these equations, as well as the unit operating ranges for this example, can be found in Table 1.

Parameter	Unit 1	Unit 2	Unit 3
Maximum	600 MW	400 MW	200 MW
Minimum	100 MW	100 MW	50 MW

Input-Output Curve:

a i	0.001562	0.00194	0.00482
b i	7.92	7.85	7.97
c i	561	310	78

Incremental Cost Curve

d i	0.003124	0.00388	0.00964
-----	----------	---------	---------

Table 1: Example Unit Data

Cost per unit fuel is assumed to be the same for all the units, therefore we may equate thermal heat content used to actual generation costs for comparison issues [1].

Objective and Fitness Functions

The economic dispatch problem can be solved by the genetic algorithm using the incremental cost curves. The curve solution uses the standard objective function and a penalty term for the conservation of energy constraint. Unit limits are automatically satisfied by a normalization process in the chosen encoding described previously, which only allows solutions in the operating range of the units.

The constraint equation can be rewritten as:

$$\text{lerr} = |\sum P_i - P_L - P_{TL}|$$

Which when reduced to zero satisfies the original constraint. The incremental cost curve representation constraint follows:

$$\frac{dF_i(P_i)}{dP_i} = \dots = \frac{dF_n(P_n)}{dP_n} = \lambda$$

This added constraint can be handled in the same fashion as the conservation of energy constraint. An error term is then introduced and a measure of this error is calculated. The constraint is met if each term in the equality is equal to the average lambda value for the string. Consequently, the error term is:

$$\lambda_{\text{err}} = \sum |\lambda_{\text{avg}} - \lambda_i|$$

where

λ_{err} = error term

λ_i = incremental cost for unit i

λ_{avg} = average incremental cost value of string

Since a chromosome string's fitness will be compared to the fitness value of all other strings within the same population, an absolute measure of optimality is not required. Likewise, the constraint equation error can be calculated as a comparison within the same population. The result is a fitness function based on a percentage rating. The following form was used:

$$\% \text{ lerr} = \frac{\text{stringerr} - \text{minerr}}{\text{maxerr} - \text{minerr}}$$

where

$\% \text{ lerr}$ = percentage of string's constraint error

stringerr = string's error in meeting constraint equation

minerr & maxerr = minimum and maximum constraint error within population

The fitness function objective then becomes:

$$\text{OBJ} = \text{MIN} [\% \lambda_{\text{err}} - \% \text{ lerr}]$$

Since the fitness function is a raw value measure of the string's fitness, we desire to give the best solutions the

highest fitness values while minimizing the function above. The minimization is changed into a maximization by folding the fitness about the value one. If algorithm control or emphasis to certain problem objectives or constraints is desired, scaling factors are added. The final form of the fitness function becomes: [1]

$$\text{Fit} = sf_1 * [(1 - \% \lambda_{\text{err}})^{(\text{sp } 1)}] + sf_2 * [(1 - \% \text{ lerr})^{(\text{sp } 2)}]$$

where

sf = scaling factor for emphasis w/in function

sp = scaling factor for emphasis over entire population

V. GENETIC ALGORITHMS WITH ELITISM

Modifications can be made to improve the performance capability of SGA. Elitism is a technique used to save early solutions by ensuring the survival of the most fit strings in each population. Elitism compares the results of the most recent population to the elite population. It then combines the two populations and determines the best results from both populations in order of decreasing fitness value. If a duplication is found, elitism eliminates this duplication. This combination of the most fit strings becomes the elite population. The process continues for each generation so that accuracy and convergence capability can be maintained in this algorithm [2].

Another modification to improve SGAs performance was the implementation of a penalty factor. This penalty factor is linear. When a deviation exists between actual power generated and the required system output, a set penalty factor is multiplied to this deviation. The greater the deviation, the greater the penalty is. The penalty function staggers the fitness function of each string in the population. The penalty function grows linearly as the number of generations increase. For example, the fitness function for three strings having the following output values and penalty factor would be:

Required System Output: 850 MW

System 1	System 2	System 3
850.2 MW	829.4 MW	857.8 MW

Penalty Factor: 1000

Fitness Function for System 1: $0.2 * (1000) = 200$

Fitness Function for System 2: $20.6 * (1000) = 20,600$

Fitness Function for System 3: $7.8 * (1000) = 7,800$

VI. REFINED GENETIC ALGORITHM

Most of the refined genetic algorithm subroutines mimic the subroutines in the SGA program, however the reproduction operators, crossover and mutation, differ between programs. Four other differences between the programs exist. They are the use of elitism and penalty factors, along with the changing probabilities of mutation and crossover occurrence.

The first difference occurs in the crossover technique. When crossover is deemed necessary, a binary string that is

the same length as the population strings is created. This string is used to cue the two parent strings as to whether the child string will get its bit values from the first or second parent. For example, if the random strings bit value in the first string position is a '1', the first parent gives its bit value to the child, however if it is a '0' the second parent donates its bit value in the first string position. This process is continued for every bit in the string. In order to create the second child, a complement procedure is followed. For example, if the random string bit value is a '0', the first parent gives its bit value to the child, however if its a '1' the second parent donates its bit value in the string. To illustrate this procedure:

Pattern String: 101001

Parent 1: 000010

Parent 2: 111001

Child 1: 010000

Child 2 (complemented child): 101011

After crossover is completed, mutation is performed. The random number generator is called to determine if mutation is necessary for the first bit in the string. If it is necessary, the present bit value is complemented at that location. For example, if the first bit in the string was originally a '1', the bit would change to a '0' during mutation. The process continues for each remaining bit in the string and for each string in the population.

RGA also uses the same elitism and penalty factor techniques found in SGAE. The final difference between RGA and SGA is the changing probabilities of mutation and crossover occurrence. In SGA, a probability of mutation and crossover is entered at the beginning of the program. This percentage remains the same throughout the entire run of the program. On the other hand, RGA changes these probabilities for each generation. Initially a probability for each is entered. For every generation thereafter, the probability of crossover is exponentially decreased while the probability of mutation is exponentially increased. Limits are set so that the probabilities do not exceed specified standards.

VII. SOLUTIONS AND PERFORMANCE

For each data set case, five separate runs were made to test the efficiency and accuracy of the program using different initial random seed numbers. The cases considered were:

- Incremental heat rate curves for SGA
- Incremental heat rate curves SGAE
- Incremental heat rate curves for RGA
- Same curves with mutation prediction
- Same curves with interval approximation
- Same curves with penalty function change
- Same curves with weighting factor change

For each case both encoding methods were tested. The initial parameters for all cases were:

- 100 - Generations
- 100 - Strings per generation
- 11 - Number of bits for string representation
- .75 - Percentage occurrence of crossover
- .01 - Percentage occurrence of mutation
- 25 - Load constraint scaling factor
- 15 - Cost scaling factor
- 3 - Scaling power factor

For each case, a table was created showing the best string solution of the final generation for all five runs. The optimal solution obtained from a verification program using classical techniques is illustrated in Table 2.

LOAD (MW)	UNIT 1 (MW)	UNIT 2 (MW)	UNIT 3 (MW)	LAMBDA
850	393.2	334.6	122.2	27.4446

Table 2: Classical Solution

Incremental Heat Rate Curves for SGA

The solution results using the SGA program are illustrated in Tables 3 and 4. Table 3 illustrates results calculated using the series encoding method, while Table 4 illustrates solutions found using the embedded encoding method. This same arrangement for the encoding schemes will be used for each case. The series will always be the first and the embedded second.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
397.02	330.679	121.519	849.218	27.4348
403.371	323.791	122.985	850.147	27.442
397.264	327.748	124.011	849.023	27.4482
393.6	335.809	122.692	852.101	27.4553
388.959	338.153	122.692	849.805	27.4499

Table 3: Series Encoding Results for SGA

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
398.73	323.791	127.382	849.902	27.4348
396.532	324.524	128.774	849.829	27.4793
397.264	330.826	121.519	849.609	27.4361
390.914	336.102	123.131	850.147	27.4523
396.043	324.963	127.528	848.534	27.4675

Table 4: Embedded Encoding Results for SGA

This SGA program found the classical optimal solution 20 - 30% of the time. The exact classical solution may never be found if the encoding does not represent the exact unit solution. Convergence can be problematic for both encoding methods.

Heat Rate Curves for SGAE

The solution results using SGAE are illustrated in Tables 5 and 6.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
393.356	334.343	122.252	849.976	27.4352
403.127	324.963	121.886	849.951	27.4446
393.112	334.636	122.252	850	27.445
393.112	334.636	122.252	850	27.445
393.112	334.636	122.252	850	27.445

Table 5: Series Encoding Results for SGAE

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
388.356	336.248	125.037	850	27.4643
392.379	334.929	122.692	850	27.4481
399.951	324.963	125.037	849.951	27.4557
397.02	330.972	122.032	849.024	27.4409
397.509	329.653	122.838	850	27.445

Table 6: Embedded Encoding Results for SGAE

The modified SGA program definitely outperformed SGA. The main difference was the programs' ability to actually converge on the classical solution. Whereas the original SGA program required an extreme number of generations to converge, the modified version converged within 30 - 40 generations. The series encoding method outperforms the embedded method.

Incremental Heat Rate Curves for RGA

The solution results using RGA are illustrated in Tables 7 and 8.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
385.54	336.248	125.55	847.338	27.4594
395.554	333.317	121.812	850.684	27.4433
389.936	335.076	125.037	850.049	27.4636
394.333	331.851	125.183	851.368	27.4662
393.6	332.534	123.351	849.536	27.4491

Table 7: Series Encoding Results for RGA

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
403.615	324.524	121.886	849.636	27.4494
392.623	334.196	121.886	848.705	27.4382
390.425	337.714	122.325	850.464	27.4492
397.509	329.8	122.032	849.34	27.4378
397.509	330.239	122.325	850.073	27.4424

Table 8: Embedded Encoding Results for RGA

It is difficult to compare the RGA and SGAE programs. Both converge to a solution within 30 - 40 generations. The SGAE series encoding method did tend to outperform the other three trials. For this reason, the SGAE series encoding program will be used exclusively as a basis for comparison for the following program enhancers, not including Mutation Prediction.

Mutation Prediction

By reducing the number of times that the random number generator is called, computer usage time can be minimized and the efficiency of the refined genetic algorithm can be maximized. Since RGA needs to call the random number generator once for every bit, in every string, in the population for each generation, much computer time is wasted. One technique, that minimizes the number of times that the random number generator needs to be called in this subroutine, was explored.

This technique uses the geometric mean probability theory. The theory states that given n independent tasks, with equal probability of occurrence, the task will occur at the $[1/\text{probability}]$ position and every bit thereafter that is a multiple of that number. For example, 15 independent tasks each have a 20% chance of occurring. According to this theory the task will occur at the $n \cdot [1/0.20] = 5\text{th}, 10\text{th}$, and the 15th positions. For every generation, the geometric mean probability theory is applied to each string in the population. The probability used for the equation is the probability of mutation. Since the probability of mutation increases exponentially for each new generation, randomization in the mutation subroutine is maintained. The solution results using the mutation prediction technique are illustrated in Tables 9 and 10.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
394.333	331.851	125.183	851.368	27.4662
393.6	332.584	123.351	849.536	27.4491
390.914	336.395	122.618	849.927	27.4485
389.936	334.783	125.403	850.122	27.466
391.891	337.274	121.446	850.611	27.4436

Table 9: Series Results for Mutation Prediction

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
393.845	334.196	121.739	849.78	27.4406
932.868	334.636	121.739	849.243	27.4393
393.112	333.61	125.037	851.759	27.4678
392.868	333.515	121.886	850.269	27.4441
391.402	333.727	121.739	846.898	27.4313

Table 10: Embedded Mutation Prediction

The results using mutation prediction resemble the results found using SGAE. The only observable difference is the amount of time it takes to converge on a solution. The modification reduces the time it takes to complete the run by approximately 50%. This modification minimizes computer time without sacrificing accuracy.

Interval Approximation

Several techniques to improve the accuracy of RGA and SGAE were also explored. The first technique involves the number of bits in the chromosome string which are used to represent the decimal value of the power output of each generator. The number of bits in the unmodified GA programs is 11. An analysis was completed to see how closely these 11 bits would decode into decimal values that

equaled the optimal output power for each generator. An example of this analysis and the results of the analysis are illustrated below:

The maximum value for 11 bits after conversion from base 2 to base 10 is 2047. In order to ensure that the output remains within specified limits, the following division and subtraction must occur.

Gen: $(P_{max} - P_{min}) = X \text{ \& } (Converted \ \# / X) = \text{Multiplying factor}$
 P1: $600 - 100 = 500 \text{ \& } 2047/500 = 4.094$
 P2: $400 - 100 = 300 \text{ \& } 2047/300 = 6.823$
 P3: $200 - 50 = 150 \text{ \& } 2047/150 = 13.647$
 Gen: $(Optimal \ Output - P_{min}) = Y$
 P1: $393.2 - 100 = 293.2$
 P2: $334.6 - 100 = 234.6$
 P3: $122.2 - 50 = 72.2$

The decimal value for each generator is then converted back into its equivalent binary value for the string.

P1: $293.2 * (2047/500) = 1200.368$
 P2: $234.6 * (2047/300) = 1600.754$
 P3: $72.2 * (2047/150) = 985.2893$

Next binary strings are created that are as close as possible to the required value.

P1: $1200 \gg 10010110000 = 1200/4.094 = 293.111$
 $1201 \gg 10010110001 = 1201/4.094 = 293.356$
 P2: $1600 \gg 11001000000 = 1600/6.823 = 234.489$
 $1601 \gg 11001000001 = 1601/6.823 = 234.636$
 P3: $985 \gg 01111011001 = 985/13.647 = 72.179$
 $986 \gg 01111011010 = 986/13.647 = 72.252$

Repeating the process for a bit with 20 strings the results become:

Base 10 Conversion: 1048575
 Multiplying Factor:
 P1: 2097.152 P2: 3495.253 P3: 6990.507
 Approximated Values & Computer Solution
 P1: $614884 \gg 293.2 \text{ \& } 614885 \gg 293.2$
 P2: $819986 \gg 234.6 \text{ \& } 819987 \gg 234.6$
 P3: $504714 \gg 72.2 \text{ \& } 504715 \gg 72.2$

Table 11: Results for Bit Length Analysis

The 11 bit program is unable to successfully converge on the exact optimal power output for each generator. This above technique was performed on several other strings with longer string lengths. With the addition of each new bit, the decimal values moved closer to being equal to the optimal values. A 20 bit string allowed for optimal values, regardless of the value of the last bit. The results using SGAE with interval approximation appear below.

Even though the SGAE program with 20 bit chromosome strings is better able to converge on the optimal solution, this did not occur without a significant increasing in the number of generations. It thus took the computer a longer time to find these results. A typical run with 11 bits took 1/2 the time to run as the 20 bit SGAE program. Programs with 20 bits took more than 100 generations to converge, versus the 30 - 40 generations

needed for the 11 bit programs. This modification does not appear to be a viable enhancement technique.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
388.715	336.248	125.037	850	27.4643
392.379	334.929	122.692	850	27.4481
399.951	324.963	125.037	849.951	27.4557
397.02	330.972	122.032	850.024	27.4409
397.309	329.653	122.838	850	27.445

Table 12: Series Interval Approximation

Consequently, from this point forward the results found that are near optimal should be considered optimal because they are as close to the true optimal solution as the computer is capable of finding with only 11 bits. These results appear below.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
393.112	334.636	122.252	850	27.445

Table 13: Approximate Optimal Results

Penalty Factor Change

Another technique to improve the accuracy of RGA and SGAE involves changing the slope of the penalty factor. At present, the penalty factor remains the same for each generation. The amount of deviation from the load is multiplied by a fixed number. When the generations progress the solution seems to oscillate above and below the optimal solution. By narrowing the slope of the penalty function as the generations progress, this problem can be alleviated. The solution results using the penalty factor modification are illustrated in Table 14.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
392.868	334.636	122.399	849.902	27.4456
393.112	334.636	122.252	850	27.445
393.356	334.343	122.252	849.951	27.4446
393.112	334.636	122.252	850	27.445
397.509	329.653	122.838	850	27.445

Table 14: Series Penalty Factor Change

This modification does seem to improve the performance capability of SGAE without increasing the time needed to run the program. This modification destroys the quick convergence found in the other SGA program. Also the narrow penalty function sometimes spends too many generations trying to reach 850 MW and not enough generations searching for the optimal generator output values.

Weighting Factor Change

The final modification combines two techniques. The first one is the above technique which modifies the penalty factor. The second one modifies the weights associated with error constraints. This change decreases the weight of the

deviation from the load constraint, while maintaining the weight of the deviation from the average objective function. The weight adjustment continues incrementally until the last iteration where the two weights are finally equal to the value of the weight of the deviation from the average objective function. The solution results using this combination of techniques are illustrated in Table 15.

UNIT 1	UNIT 2	UNIT 3	TOTAL	LAMBDA
393.112	334.636	122.252	850	27.445
393.112	334.636	122.252	850	27.445
393.112	334.636	122.252	850	27.445
393.112	334.636	122.252	850	27.445
393.112	334.636	122.252	850	27.445

Table 15: Series Weighting Factor Change

The combination of the two techniques seems to be a successful approach to increase the convergence accuracy of SGAE. The series method found the closest achievable values to the optimal solution 100% of the time. The only negative aspect of the combination is that early convergence is sacrificed. It takes 70 - 80 generations to converge on the optimal solution.

Using an IBM Personal Computer (Model 8513001), the program took about 63 seconds to solve the ED problem. This is a considerable amount higher than the techniques currently being used today which take 10 to 20 seconds to solve this problem. However, one advantage of the GA approach is that it is able to solve highly nonlinear and discontinuous problems without the approximations needed in the classical techniques.

VIII. CONCLUSIONS

Elitism is an effective tool to improve the performance capability of genetic algorithms. It also helps illustrate a possible method to calculate the "stopping point" for the program. Because elitism stores the fittest strings from each population, the programs are able to quickly find and keep the best solutions to the problem. When the program converges, it produces a natural stopping criteria for the program. The program can stop after the elite population is not updated for five (5) generations, computer usage time can be drastically reduced. The exact method merits further study.

Most of the enhancement techniques improved the efficiency and/or accuracy of RGA and SGAE. The only technique that does not seem to be beneficial is the interval approximation technique. However, a new technique which slowly increases the number of bits, while freeing the previous bits, is under investigation. The combination of the penalty factor and weighting modifications appears to be the most beneficial combination. Although it requires an increased number of generations to converge on an optimal solution, the results are so greatly improved that it makes the sacrifice worthwhile.

Consequently, if mutation prediction is combined with the other two, the computer time would be reduced so the first sacrifice would no longer need to be considered. Accuracy and efficiency could be achieved without sacrificing

anything. Finally, with the use of multiprocessors, the time it takes a genetic-based program to solve the EDC program can be drastically reduced and can become easily comparable to the time it takes to solve the problem using classical approaches.

The exact determination of a correct solution is dependent on the expertise of the user. The user must be familiar with previous solutions and typical convergence rates. It would be incorrect to assume that a solution calculated in 95 generations is wrong unless background information supported this assumption. Expertise with unique programs and problems would enable the user to clearly decipher between a correct and incorrect answer.

REFERENCES

1. Walters, David, C., Sheble', Gerald, B. "Genetic Algorithm Solution of Economic Dispatch with Valve Point Loading," IEEE Transactions on Power Systems, Vol PWRS 414 - 3.
2. Goldberg, D.E., Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, Reading, MA, 1989.
3. Walters, David C., An Application of Genetic Algorithms to the Economic Dispatch of Power Generation, MS Thesis, Auburn University, May, 1991.
4. Gross, C.A., Power System Analysis. John Wiley & Sons, New York, NY, 1986.
5. Noyoloo, A.H., Grady, W.M., Viviani G.L. "An Optimized Procedure for Determining Incremental Heat Rate Characteristics," IEEE Transactions on Power Systems, Vol. 5, May 1990, pp. 376-383.
6. Wood, A. J. and Wollenberg, B.F., Power Generation, Operation, and Control, John Wiley & Sons, New York, NY 1984.

Gerald B. Sheble' (M 71, SM '85) is an Associate Professor of Electrical Engineering, Iowa State University, Ames, Iowa. Dr. Sheble' received his B.S. and M.S. degrees in Electrical Engineering from Purdue University and his Ph. D. in Electrical Engineering from Virginia Tech. His research interests include distribution and power system optimization, scheduling and control.

Kristin Brittig completed her B. S. in Electrical Engineering at Iowa State University. Her interests include Power and Control Systems Analysis and Optimization.

Discussion

L.L. Lai and F. Ndeh-Che, (Energy Systems Group, City University, London EC1V 0HB, England, UK): Genetic Algorithms (GAs) make use of an internal representation of a given system in order to perform optimisation functions. The actual structural layout of this representation, called a genome, has a crucial impact on the outcome of the optimisation process. The authors are to be congratulated for an interesting paper to study the effects due to different internal representation in a GA.

Though the field of natural genetics is progressing quite rapidly, understanding of the genetic process is still quite incomplete. Natural genetic evolution is capable of adding new functionality to species represented by relatively stable genetic material. How new genetic materials will be added to the population may vary, but one method consists of duplicating a section of existing materials. Once this material has been duplicated, variations of it will result in the exhibition of the new functionality. The representation of this new functionality in the gene can be based on the representation of the original functionality. But this representation should be able to vary in order to find a more appropriate representation and to survive as stable genetic material.

How could these alternative representation be generated and tested is far from being fully understood? Both representation descriptions and actual function descriptions must co-exist in the same genetic material. As in natural selection, the individuals with better fitness have a higher probability of reproducing and therefore maintaining their genetic material in subsequent generations. The discussor believes that one important issue is that the range of fitness values and their effectiveness is related and sensitive to the size of initial population.

By restricting the population size, the genetic operators may loose the optimal genetic material by placing it with sub-optimal material and eliminating it within the next generation. It is important to have enough genetic material and combinations of material in the initial population to examine the entire solution space in detail. It would be appreciated if the authors could explain

the study for different population size, if any?

Manuscript received March 4, 1994.

G. B. Sheble': The authors thank the reviewers for their comments. The issues raised by the authors are very pertinent to the quickly changing field of artificial reality. The reviewers are correct that genetic material must remain beyond a single generation. We have found that "elitism," keeping a single solution beyond one generation, is one approach to reducing the population size. The natural analogy is the maintenance of dormant information within the genetic structure.

We have studied the impact of population size. Based on the selection process of mating throughout the population, the less fit genetic material is always eliminated within the first one or two generations. As the reviewers have noted, this inferior genetic material is quickly eliminated. This is analogous to the selection of a mate across an entire continent and not just the local domain. Selection of mates within a local domain is presently under investigation.

We have also studied the impact of the randomness of the initial population. Alternate seeds were used to generate the initial population. The initial individuals were mapped into the solution space and a euclidean measure of the space cover was calculated. It was found that the solution space was covered rather "uniformly." That is, the spheres generated by the euclidean distance measure would have provided good results for most clustering techniques. The impact of such a finding on the probability of covering the entire solution space is unknown at this time.

Manuscript received April 18, 1994.